



GUESS THE SONG



שם בית הספר: תיכון חוגים
שם העבודה
שם התלמיד: ניר שדה

שם המנחה: חגית כהן
שם החלופה: תכנות טלפונים חכמים
תאריך הגשה: 15.6.2025

תוכן עניינים

3	מבוא
3	שם הפרויקט
3	קהל היעד
3	הסיבות לבחירת הנושא
3	מטרת המשחק
3	אופן המשחק
3	סיום המשחק
4	דרישות מינימליות
4	שימוש בספריות
4	טכנולוגיות
4	אתגרים מרכזיים
5	תהליך המחקר
6	תיאור תחום הידע - פרק מילולי
6	אובייקטים נחוצים
6	סוגי נתונים
6	ייצוג מידע
6	תיאור פעולות על המידע
7	מבנה/ ארכיטקטורה
7	MainActivity - מסך כניסה
8	OptionsActivity - מסך בחירת הקטגוריות והערכים
9	SettingsActivity - מסך הגדרות
10	PlayActivity - מסך המשחק
12	תרשים UML
12	דיאגרמת מסכים
13	תיאור פנימי
14	מימוש הפרויקט
14	מחלקת Song
14	תפקיד המחלקה
14	תכונות המחלקה
15	פעולות המחלקה
17	מחלקת Songs
17	תפקיד המחלקה
17	תכונות המחלקה
17	פעולות המחלקה
19	מחלקת FirebaseLogic
19	תפקיד המחלקה
19	תכונות המחלקה
19	פעולות המחלקה
24	מחלקת SoundService
24	תפקיד המחלקה:
24	תכונות המחלקה:
24	פעולות המחלקה:

Guess The
Song

25
25
25
26
28
29
31
32
33

ניר שדה

מחלקת SaveData
תפקיד המחלקה
תכונות המחלקה
פעולות המחלקה

מסד נתונים
מדריך למשתמש
רפלקציה
ביבליוגרפיה
נספחים

מבוא

שם הפרויקט

Guess The Song

קהל היעד

המשחק מיועד לכל מי שאוהב מוזיקה ורוצה לבחון את הידע שלו לפי זיהוי שיר לפי המילים או קטע אודיו מהשיר.

הסיבות לבחירת הנושא

בחרתי לעשות משחק שמבוסס על מוזיקה משום שאני מאוד אוהב לשמוע מוזיקה ורציתי לבחון את עצמי עד כמה אני מסוגל לזהות שירים לפי המילים שלהן או חלק שמע מהם.

מטרת המשחק

על המשתמש לנחש נכון כמה שיותר שירים מהקטגוריה שהוא בחר עד שלא ישארו עוד שירים מהקטגוריה.

אופן המשחק

תחילה, המשתמש צריך לבחור לפי איזו קטגוריה הוא רוצה לנחש שירים: ז'אנר מוסיקלי, אמן או עשור. לאחר שהמשתמש בוחר את הקטגוריה הוא צריך לבחור אופציה מהקטגוריה שבחר לדוגמה: ז'אנר רוק אלטרנטיבי מבין סוגי הז'אנרים או קווין מבין אפשרויות האמנים.

סיום המשחק

המשחק נגמר כאשר המשתמש מסיים לנחש את כל השירים מכל הקטגוריות.

דרישות מינימליות

API 30

טכנולוגיות

Firestore - האפליקציה שומרת באמצעות מסד נתונים בענן את הפרטים של המשתמשים באפליקציה ובנוסף יוצרת עבור כל אחד מהם אוסף של השירים שעליהם לנחש כשבעבור כל שיר מוצג האם אותו משתמש ניחש אותו או שלא

SharedPreferences - במהלך המשחק, המשתמש בוחר את הקטגוריה והערך שלה שלפיה הוא רוצה לנחש שירים, כך שגם כאשר יועבר למסך בו הוא מנחש את שם השיר, הערכים הללו ישמרו לצורך נוחות כך שהוא יוכל להמשיך לנחש שירים מאותה קטגוריה גם כשיחזור למסך הבחירה.

MediaPlayer - כאשר המשתמש מנחש שירים על פי אודיו מועמדים לרשותו שלושה כפתורים שבאמצעותם הוא שומע את קובץ האודיו - כפתור הפעלה, כפתור עצירה וכפתור החזרה לאחור ובנוסף, כאשר השחקן עובר לנחש את השיר הבא, קובץ האודיו מוחלף.

אתגרים מרכזיים

1. על מנת להעלות את בסיס הנתונים לענן, הייתי צריך ליצור אוסף בשם Master, כשניסיתי להעלות אותו בהתחלה זה לא עובד כתוצאה מירושא לא נכונה של המחלקות אך לאחר מכן, בסיס הנתונים עלה לענן.
2. כאשר שחקן ניחש את שם השיר כראוי, הערך של השיר במסד הנתונים של אותו משתמש היה צריך להיות להשתנות לtrue, אך זה לא קרה כתוצאה משגיאת הקלדה בקוד שמשנה את הערך במסד הנתונים.
3. על מנת שקובץ השמע שיושמע כאשר השחקן רוצה לנחש שיר יהיה תואם לשם השיר, הייתי צריך לחקור איך לוודא ששם הקובץ שהוא אחד מהתכונות של שיר, יופיע כראוי בשורה שמנגנת את הקובץ.
4. תחילה, רציתי שהשחקן יוכל לנחש שירים על פי עשורים, אך הדבר לא הסתדר משום שלא הצלחתי להוציא ממסד הנתונים שבענן שירים בטווח שנים מסוים עד שלבסוף החלטתי שהשחקן יוכל לבחור לנחש שירים החל משנת 1970 עד שנת 2019.

תהליך המחקר

לצורך הפרויקט חיפשתי משחקים שכבר קיימים שעוסקים באותו הנושא - ניחוש שירים על פי המילים או קטע אודיו, מצאתי שני משחקים דומים, האחד בשם Bandle שצריך לנחש את שם השיר על פי כלי

הנגינה של אותו שיר עד שלא נותרו עוד כלים ואז נחשף השיר ובנוסף ישנו משחק של עיתון הארץ ובו צריך לנחש בכל יום את המילים של שיר לועזי שמתורגמות לעברית. יתר על כן, למדתי כיצד לעבוד עם מסד נתונים בענן - Firebase וכיצד להפעיל קובץ אודיו על פי המידע שנשלף מהענן.

חידושים

- שימוש במסד הנתונים Firebase על מנת לשמור עבור כל משתמש את פרטיו: אימייל וסיסמה ואת הנתונים האישיים שלו במשחק - השירים שהוא ניחש ואלו שעדיין לא.
- שימוש בנגן MediaPlayer על מנת להשמיע קבצי אודיו מסוג MP3.

תיאור תחום הידע - פרק מילולי

אובייקטים נחוצים

שירים - השירים שעל השחקן לנחש לפי המילים שלהם או קובץ אודיו, ושאר המידע עליהם.
משתמשים - כל אדם שרוצה לשחק צריך להתחבר לאפליקציה באמצעות כתובת אימייל וסיסמה, או ליצור משתמש משל עצמו.

סוגי נתונים

נתוני המשתמש - כתובת אימייל, סיסמה
נתוני השירים - שנת השיר, שם האמן, ז'אנר השיר, שם קובץ השמע, האם הוא נוחש, שם השיר, מילות השיר

ייצוג מידע

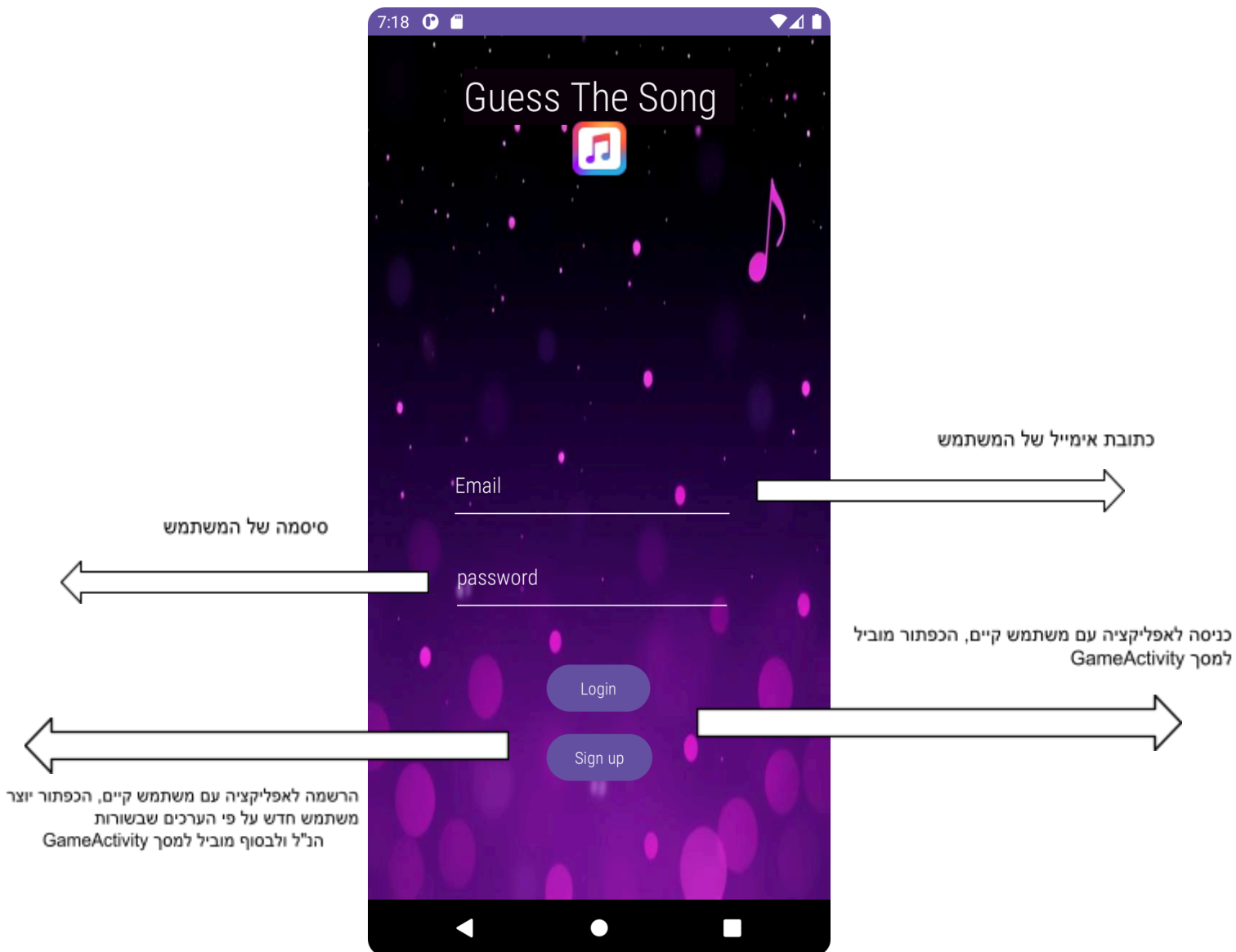
במסד הנתונים שמורים כ-60 שירים, במהלך המשחק השירים שמתאימים לערך שהמשתמש רוצה לנחש לפיו את שמות השירים, מוצגים באמצעות רשימה (ArrayList), משום שאין הגבלה של מספר האובייקטים שניתן להכניס במבנה הנתונים הזה וכאמור, אל תוך הרשימה מוכנסים השירים שמתאימים לקטגוריה ושעדיין לא נוחשו על ידי המשתמש.

תיאור פעולות על המידע

יצירת משתמש חדש - הוספת משתמש חדש
עדכון שיר - עדכון האם השיר נוחש
אתחול המשחק - עדכון כל השירים כך שהם עדיין לא נוחשו

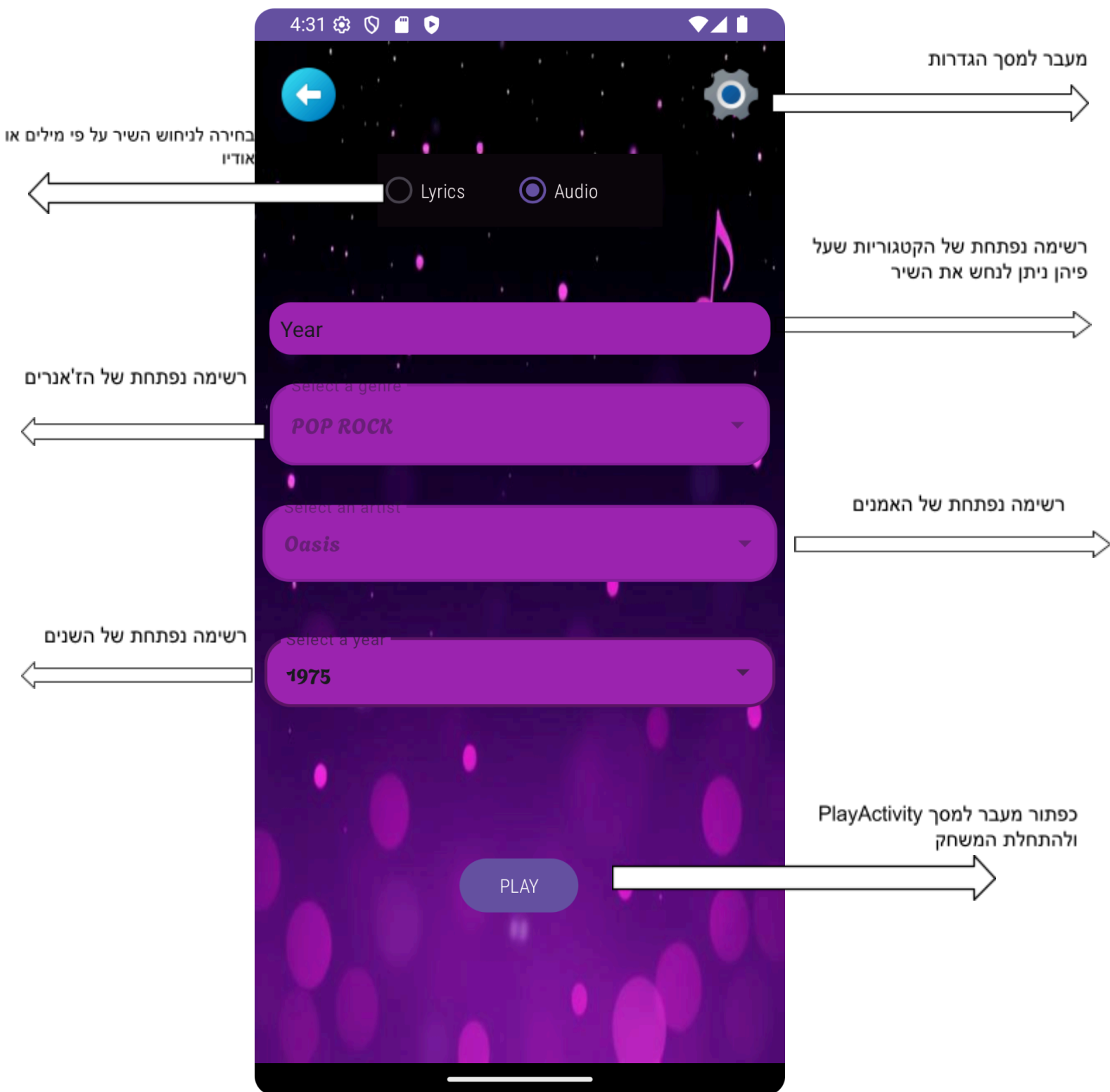
מבנה/ ארכיטקטורה

MainActivity - מסך כניסה



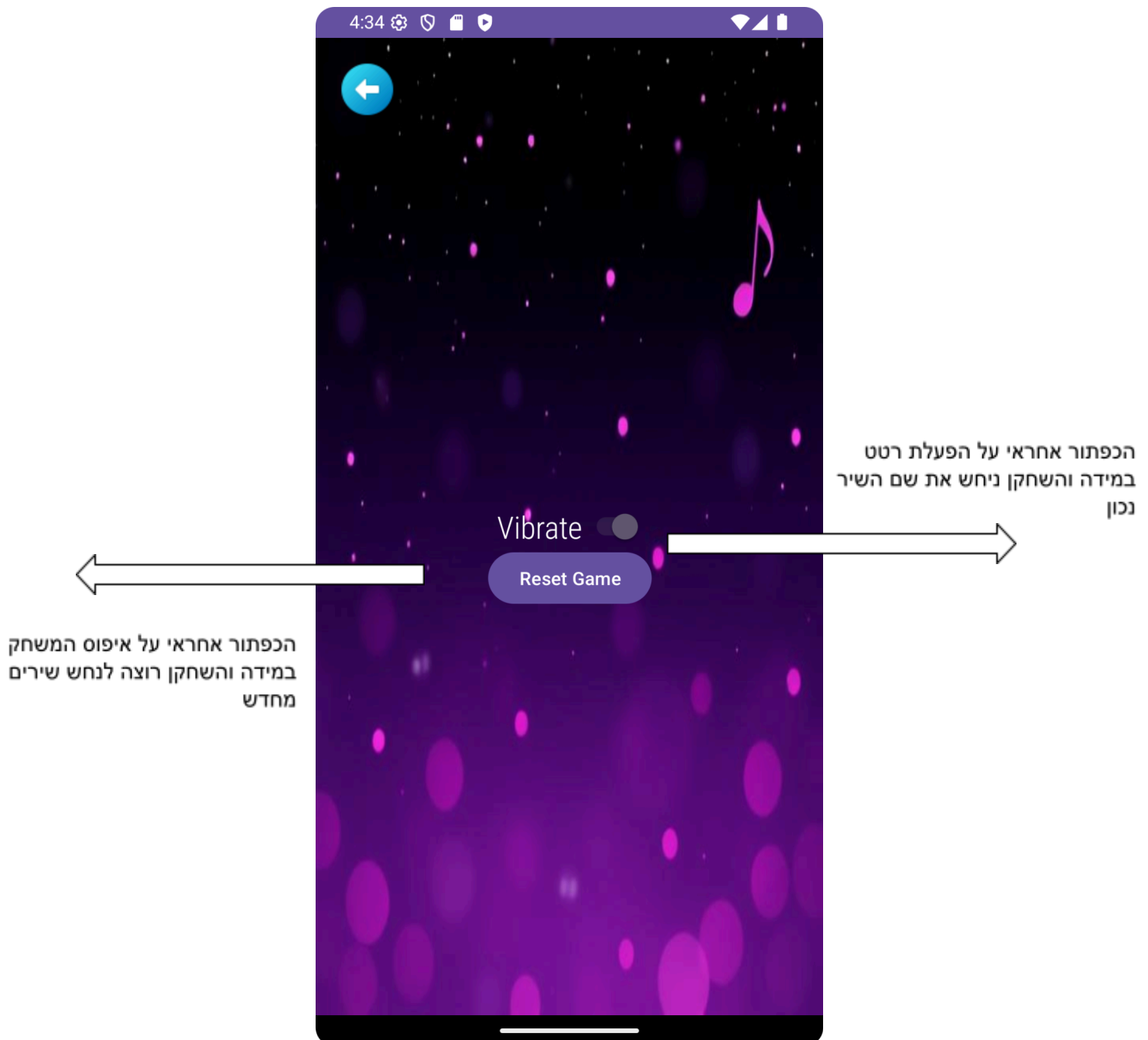
במסך הכניסה לאפליקציה ניתן להתחבר עם משתמש קיים או ליצור משתמש אחד, שתי האפשרויות מעבירות למסך הMainActivity.

OptionsActivity - מסך בחירת הקטגוריות והערכים



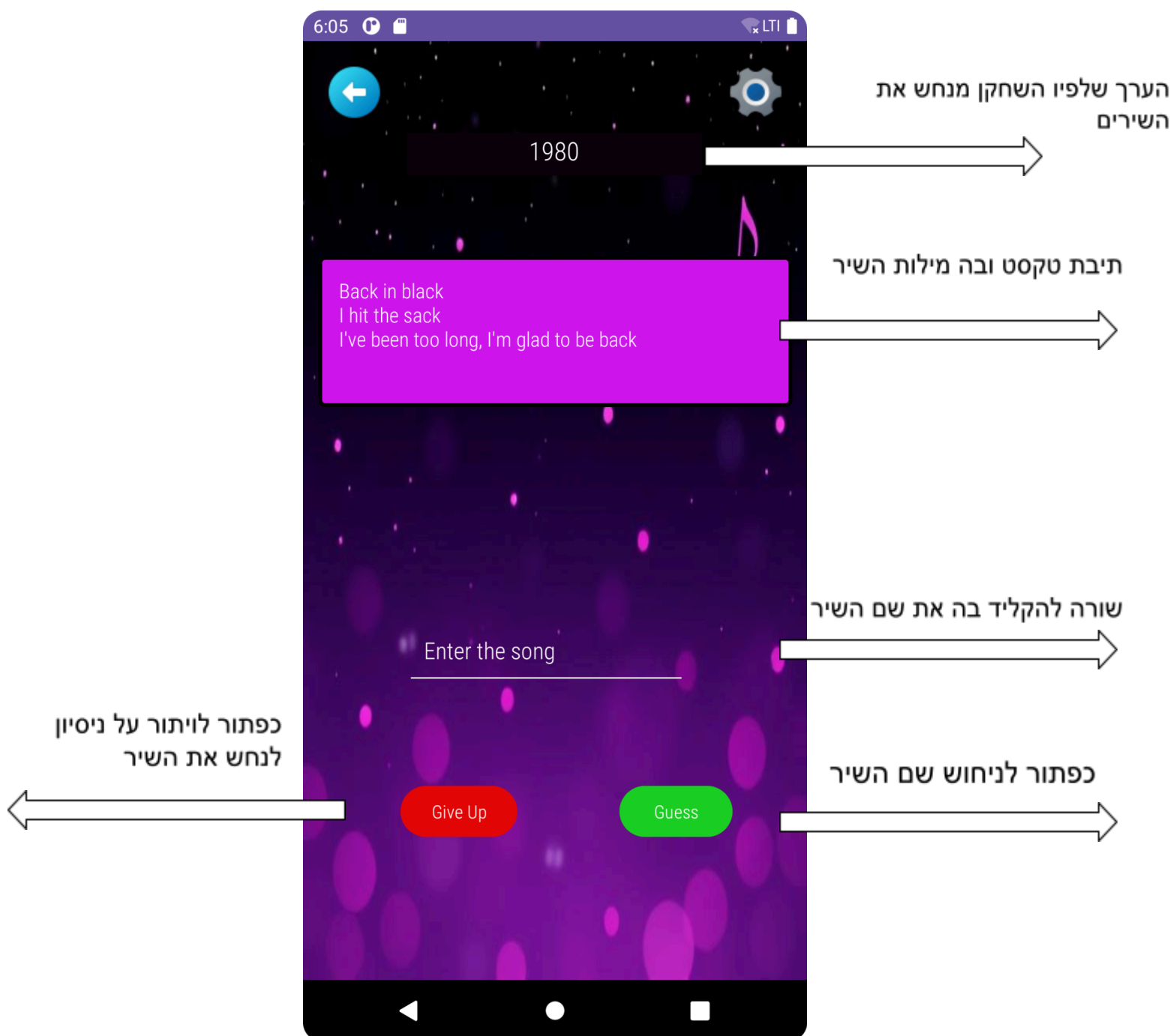
במסך זה יש לבחור את הקטגוריה שלפיה השחקן ינחש את השיר: ז'אנר, אמן או עשור ואת המידע של אותו שיר לפיו השחקן ינחש: אודיו או מילות השיר

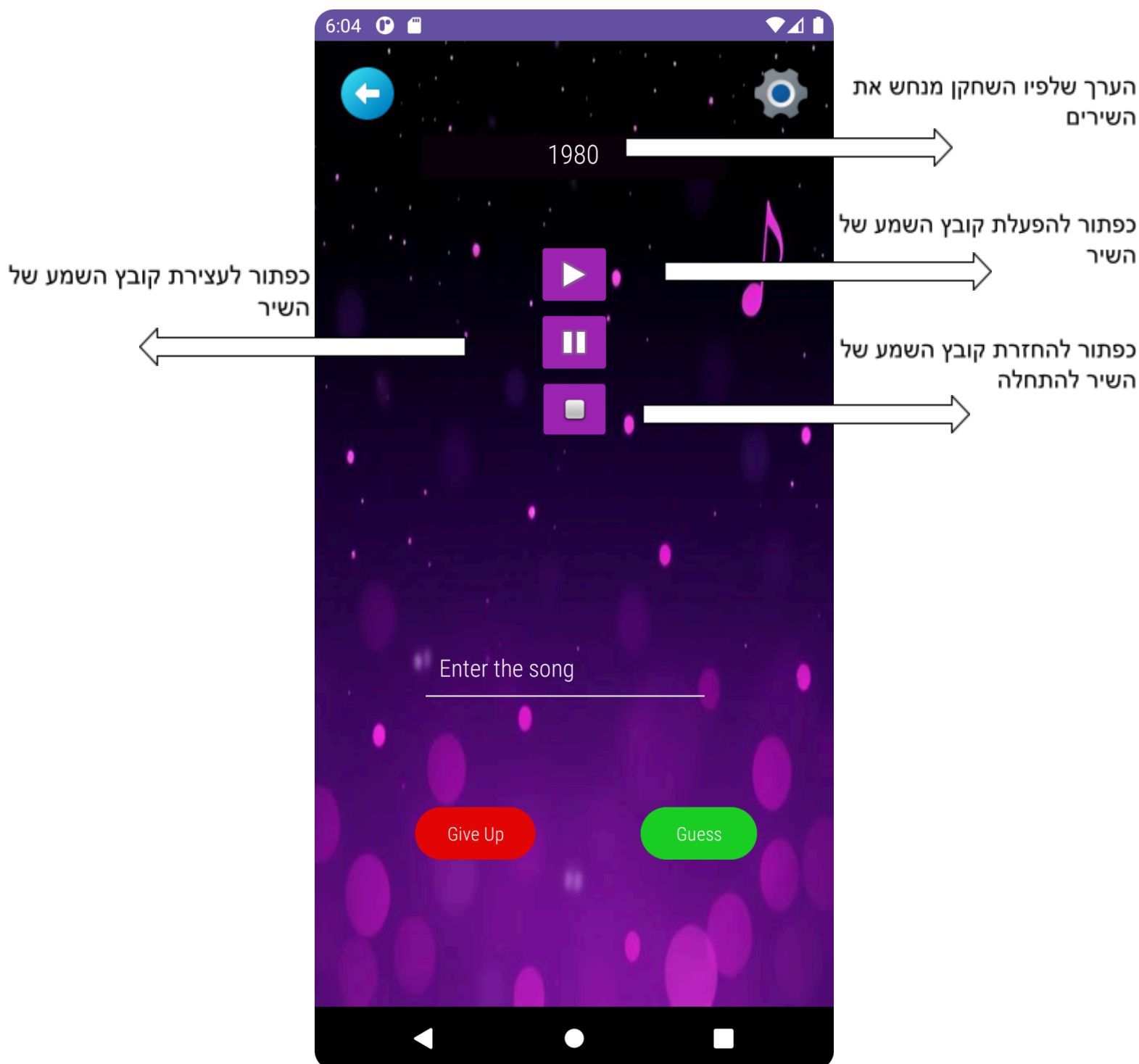
SettingsActivity - מסך הגדרות



המסך אחראי על הפעלת רטט לפי החלטתו של השחקן במידה והוא ניחש נכונה שם של שיר במהלך המשחק ועל הפעלת סאונד במהלך הכניסה לאפליקציה. בנוסף, ניתן "לאפס" את המשחק ולהתחיל לנחש מחדש את השירים על ידי שינוי כל השירים במסד הנתונים למצב שהם לא נחשו.

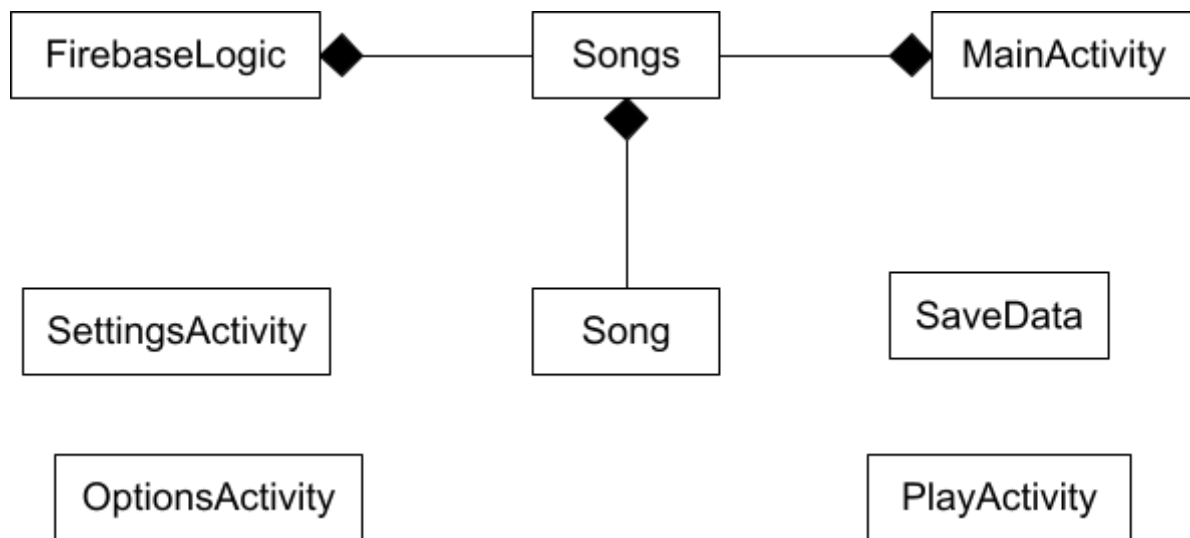
PlayActivity - מסך המשחק



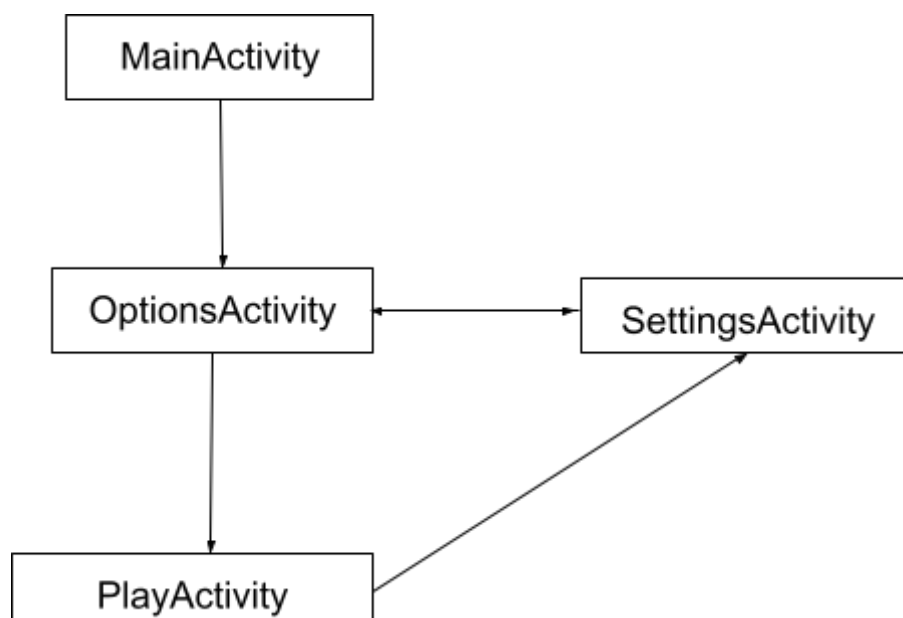


למשך המשחק יש שתי אפשרויות שבאמצעותן אפשר לנחש את השיר: או על ידי שמיעה של מספר שניות מהשיר או על ידי צפייה בחלק ממילות השיר, המשתמש מקליד את שם השיר בתיבת הטקסט, אם הוא צודק, תקפוץ הודעה בהתאם והמשחק יעבור לשיר הבא, אם הוא טועה, הוא יוכל לנחש שוב. במידה והוא יחליט לוותר, האפליקציה תראה את שם השיר ותעבור לשיר הבא עד שיגמרו כל השירים שיש לנחש בקטגוריה שהמשתמש בחר.

תרשימי UML



דיאגרמת מסכים



תיאור פנימי

```
</> activity_game.xml  
</> activity_main.xml  
</> activity_play_activity.xml  
</> activity_settings.xml
```

```
© FirebaseLogic  
© MainActivity  
© OptionsActivity  
© PlayActivity  
© SaveData  
© SettingsActivity  
© Song  
© Songs  
© SoundService
```

מימוש הפרויקט

Song מחלקת

תפקיד המחלקה

המחלקה אחראית על בניית עצם מסוג שיר והיא שומרת עבור כל עצם תכונות הרלוונטיות לשיר כדוגמת האמן של השיר, שנה, ז'אנר מוזיקלי, שם וכד'. בנוסף, ישנן פעולות שאחראיות על המרת עצם מסוג שיר לעצם מסוג Map, על מנת שיועבר כראוי למסד הנתונים.

תכונות המחלקה

שם התכונה	פירוט
private String name	שם השיר
private int year	שנת השיר
private String genre	הז'אנר של השיר
private String fileNameAudio	שם קובץ השמע של השיר
private String lyrics	מילות השיר
private String artist	שם האמן של השיר
private boolean isGuessed	האם השיר נחש על ידי השחקן או לא

פעולות המחלקה

שם הפעולה	פירוט
public Song(String name, int year, String genre, String fileName, (String lyrics, String artist	פעולה בונה שמקבלת את הערכים של השיר ובונה עצם חדש של שיר שעדיין לא נחש
public Song(String name, int year, String genre, String fileName, String lyrics, String artist, boolean isGuessed)	פעולה בונה שמקבלת את הערכים של השיר ובונה עצם חדש של שיר
public SongMap<String, Object> map ()	הפעולה מקבלת עצם מסוג Map וממירה את הערכים שלו לטיפוסי הערכים של עצם מסוג Song
public Map<String, Object> getMap()	הפעולה בונה עצם מסוג Map באמצעות הערכים של עצם מסוג Song
public Song()	הפעולה בונה עצם מסוג Song מבלי לקבל ערכים

פירוט פעולות לעומק:

```
public Song(Map<String, Object> map)
{
    this.name = (String) map.get("name");
    this.year = (int) map.get("year");
    this.genre = (String) map.get("genre");
    this.fileAudioName = (String) map.get("fileAudioName");
    this.lyrics = (String) map.get("lyrics");
    this.artist = (String) map.get("artist");
    this.isGuessed = (boolean) map.get("isGuessed");
}

public Map<String, Object> getMap()
{
    Map<String, Object> map = new HashMap<>();
    map.put("name" , name);
    map.put("year" , year);
    map.put("genre" , genre);
    map.put("fileAudioName" , (fileAudioName.toLowerCase()).replace(" ", "_"));
    map.put("lyrics" , lyrics);
    map.put("artist" , artist);
    map.put("isGuessed" , isGuessed);
    return map;
}
```

מחלקת Songs

תפקיד המחלקה

המחלקה אחראית על יצירת מערך-רשימה של עצמים מטיפוס Song ולמחיקה והוספה של עצמים בהתאם למהלך המשחק

תכונות המחלקה

שם התכונה	פירוט
private ArrayList<Song> songs	מערך רשימה מטיפוס Song

פעולות המחלקה

שם הפעולה	פירוט
public Songs()	הפעולה יוצרת מערך-רשימה ריק
public void addSongs(Song s)	הפעולה מוסיפה למערך רשימה את העצם s
public void removeSong(int number)	הפעולה מחזירה את גודל המערך-רשימה
public void create()	הפעולה מוסיפה שירים למערך-רשימה
(public void removeSong(int number	הפעולה מקבלת מספר של שיר במערך-רשימה ומסירה אותו
public Song getSong(int number)	הפעולה מקבלת מספר של שיר במערך ומחזירה אותו

פירוט פעולות לעומק:

```
public void create() {
    songs.add(new Song("Space Oddity", 1969, "ROCK", "Space Oddity", "Ground
control to Major Tom\nTake your protein pills and put your helmet on", "David
Bowie"));
    songs.add(new Song("Starman", 1972, "ROCK", "Starman", "Didn't know what
time it was, the lights were low\nI leaned back on my radio\nSome cat was
layin' down some rock 'n' roll\nLotta soul, he said", "David Bowie"));
    songs.add(new Song("Let's Dance", 1983, "ROCK", "Lets Dance", "Let's
dance\nPut on your red shoes and dance the blues", "David Bowie"));

    songs.add(new Song("Wonderwall", 1995, "BRITPOP", "Wonderwall", "Today is
gonna be the day that they're gonna throw it back to you\nBy now, you
should've somehow realised what you gotta do\nI don't believe that anybody
feels the way I do about you now", "Oasis"));
    songs.add(new Song("Supersonic", 1994, "BRITPOP", "Supersonic", "I need to
be myself\nI can't be no one else", "Oasis"));
    songs.add(new Song("Don't Look Back in Anger", 1995, "BRITPOP", "Dont Look
Back in Anger", "Slip inside the eye of your mind\nDon't you know you
might\nfind A better place to play", "Oasis"));
    songs.add(new Song("Live Forever", 1994, "BRITPOP", "Live Forever", "Maybe
I don't really want to know\nHow your garden grows, 'cause I just want to
fly", "Oasis"));

    songs.add(new Song("Lithium", 1991, "GRUNGE", "Lithium", "I'm so happy
'cause today I've found my friends\nThey're in my head", "Nirvana"));
    songs.add(new Song("Come as You Are", 1991, "GRUNGE", "Come as You Are",
"Come as you are, as you were\nAs I want you to be", "Nirvana"));
    songs.add(new Song("Smells Like Teen Spirit", 1991, "GRUNGE", "Smells Like
Teen Spirit", "Load up on guns, bring your friends\nIt's fun to lose and to
pretend", "Nirvana"));
    songs.add(new Song("About a Girl", 1989, "GRUNGE", "About a Girl", "I need
an easy friend\nI do, with an ear to lend", "Nirvana"));
```

מחלקת SoundService

תפקיד המחלקה

המחלקה אחראית על הפעלת אודיו כאשר המשתמש נכנס לאפליקציה במסך הכניסה ומופעל אודיו בהתאם להאם הצליח להיכנס או שלא.

תכונת המחלקה

שם התכונה	פירוט
private static SoundPool soundpool_	משתנה שבאמצעותו מושמע קובץ האודיו
public static int playOk;	משתנה מסוג שלם שבאמצעותו מופעל קובץ אודיו מסוים
public static int playWrong	משתנה מסוג שלם שבאמצעותו מופעל קובץ אודיו מסוים
private static float volume	עוצמת הסאונד של הקובץ המתנגן

פעולות המחלקה

שם הפעולה	פירוט
public static void setValue(Context context)	הפעולה מקבלת תוכן מסוים ומאתחלת את המשתנים לקובץ אודיו מסוים
public static void playSoundWrong()	הפעולה משמיעה סאונד ספציפי כשמשתמשים בה
public static void playSoundOk()	הפעולה משמיעה סאונד ספציפי כשמשתמשים בה

מחלקת SaveData

תפקיד המחלקה

המחלקה אחראית על שמירת מידע במסכים ספציפיים גם לאחר שיוצאים מהם כדוגמת מסך הגדרות

תכונות המחלקה

שם התכונה	פירוט
private static SharedPreferences sp	משתנה שתפקידו לשמור את הנתונים בכל מסך בצורת key-value
private static SharedPreferences.Editor edit	המשתנה עורך את הערכים של המשתנה sp
private static final String VALUE_KEY = "value"	משתנה בעל ערך קבוע של הערך שהשחקן בחר בצורת key-value
private static final String CATEGORY_KEY = "category"	משתנה בעל ערך קבוע של הקטגוריה שהשחקן בחר בצורת key-value
private static final String AUDIO_OR_LYRICS_KEY = "audioOrLyrics";	משתנה בעל ערך קבוע של הערך שהשחקן בחר בצורת key-value
private static String category = ""	הקטגוריה שהשחקן בחר
private static String value = "";	הערך שהשחקן בחר מהקטגוריה שנבחרה
private static String audioOrLyrics = "audio";	הערך לפיו השחקן רוצה לנחש את השיר: מילים או אודיו
private static final String VIBRATE_KEY = "vibrate";	משתנה בעל ערך קבוע של key של האם המכשיר ינוע
private static boolean vibrate	ערך שלפיו המכשיר ינוע בהתאם
private static final String YEAR_VALUE_KEY = "year";	משתנה בעל ערך קבוע של ערך key ב-year

private static final String GENRE_VALUE_KEY = "genre";	משתנה בעל ערך קבוע של ערך key בgenre
private static final String ARTIST_VALUE_KEY = "artist";	משתנה בעל ערך קבוע של ערך key בartist
private static String year_value = " ";	ערך השנה שהשחקן בוחר
private static String genre_value = " ";	ערך הז'אנר שהשחקן בוחר
private static String artist_value = " ";	ערך האמן שהשחקן בוחר
private static final String SOUND_KEY = "sound";	משתנה בעל ערך קבוע של ערך key בsound
private static boolean sound ;	ערך הסאונד שהשחקן בוחר

פעולות המחלקה

שם הפעולה	פירוט
public static void init(Context context)	הפעולה מקבלת את ערך value של כל key כמו שנשמר במשתנה sp
public static void setValue(String value)	הפעולה מקבלת ערך מהקטגוריה שהשחקן בחר ומציבה אותה במשתנה
public static void setCategory(String category)	הפעולה מקבלת קטגוריה שהשחקן בחר ומציבה אותה במשתנה
public static void setAudioOrLyrics(String choice)	הפעולה מקבלת את הערך שהשחקן בחר לנחש לפיו את השיר: אודיו או מילים ומציבה את הערך במשתנה המתאים
public static void setVibrate(boolean	הפעולה מקבלת ערך אמת או שקר ומציבה אותו

vibrate)	במשתנה שלפיו המכשיר ינוע בהתאם
public static void setYearValue(String value)	הפעולה מציבה ערך חדש לשנה
public static void setGenreValue(String value)	הפעולה מציבה ערך חדש לז'אנר
public static void setArtistValue(String value)	הפעולה מציבה ערך חדש לאמן
public static void setSound(boolean sound)	הפעולה מציבה ערך חדש לסאונד

פירוט פעולות לעומק:

```
package com.nirs.guessthesong;
import android.content.Context;
import android.content.SharedPreferences;

public class SaveData
{
    private static SharedPreferences sp;
    private static SharedPreferences.Editor edit;

    private static final String VALUE_KEY = "value";
    private static final String CATEGORY_KEY = "category";
    private static final String AUDIO_OR_LYRICS_KEY = "audioOrLyrics";
    private static String category = "";
    private static String value = "";
    private static String audioOrLyrics = "audio";
    private static final String VIBRATE_KEY = "vibrate";
    private static boolean vibrate;

    public static void init(Context context)
    {
        sp = context.getSharedPreferences("data", Context.MODE_PRIVATE);
        edit = sp.edit();
        category = sp.getString(CATEGORY_KEY, "");
        value = sp.getString(VALUE_KEY, "");
        audioOrLyrics = sp.getString(AUDIO_OR_LYRICS_KEY, "audio");
        vibrate = sp.getBoolean(VIBRATE_KEY, true);
    }
}
```

מחלקת FirebaseLogic

תפקיד המחלקה

המחלקה אחראית על הפעולות שבהן מועבר מידע מהענן (FireBase) אל האפליקציה או להפך. בנוסף, ישנן פעולות שמסננות את המידע מהענן לפי בקשות הלקוח.

תכונות המחלקה

שם התכונה	פירוט
private static FirebaseAuth mAuth	מכיל פרטי זיהוי של המשתמש הנוכחי במשחק
public static Songs songs	מערך של שירים
private static String uid	מחרוזת זיהוי של השחקן הנוכחי
private static Context context	תוכן שמועבר לשרת בענן כשנשלחת בקשה למידע מהענן
public static final String LOAD = "Load songs"	מחרוזת שמועברת לשרת בעת בקשת מידע

פעולות המחלקה

שם הפעולה	פירוט
public static void init(Context c)	הפעולה
public static void readUserData()	הפעולה קוראת את המידע מאוסף Users של משתמש קיים אל תוך מערך השירים
public static void readAllData()	הפעולה קוראת את המידע מאוסף Master ויוצרת אוסף חדש למשתמש חדש תחת האוסף Users
public static void writeAllData(ArrayList<Map> list)	הפעולה מעלה עבור משתמש ספציפי את מערך השירים באוסף Users בענן
public static void writeAllData(Songs	הפעולה מעלה את מערך השירים לענן תחת שם

songs)	האוסף Master
public static void getSongsByGenre(String genreType)	הפעולה מקבלת ז'אנר מוזיקלי ומאתחלת את מערך השירים לשירים מאותו ז'אנר שלא נוחשו על ידי השחקן
public static void getSongsByArtist(String artistName)	הפעולה מקבלת שם של אמן ומאתחלת את מערך השירים לשירים של אותו אמן שלא נוחשו על ידי השחקן
public static void getSongsByYear(int year)	הפעולה מקבלת שתי שנים ומוספיה למערך השירים שאת השירים מאותו טווח שנים שלא נוחשו על ידי השחקן
public static void markSongAsGuessedForUser(String songName)	הפעולה מקבלת שם של שיר שהשחקן ניחש ושולחת בקשה לענן לעדכן שהשיר נוחש
public static void resetGame()	הפעולה עוברת על מסד הנתונים של השחקן ומשנה את הערך של כל השירים לfalse בkey isGuessed

פירוט פעולות לעומק:

```
public class FirebaseLogic
{
    public static final String LOAD = "Load songs";
    private static FirebaseAuth mAuth;
    public static Songs songs;
    private static String uid;
    private static Context context;

    public static void init(Context c)
    {
        mAuth = FirebaseAuth.getInstance();
        context = c;
    }
}
```

```

public static void readUserData()
{
    uid = mAuth.getCurrentUser().getUid();
    ArrayList<Map> list = new ArrayList<>();
    songs = new Songs();
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("Users").document(uid).collection("songs")
        .get().addOnCompleteListener(new
OnCompleteListener<QuerySnapshot>()
    {
        @Override
        public void onComplete(@NonNull Task<QuerySnapshot> task)
        {
            if(task.isSuccessful())
            {
                for(QueryDocumentSnapshot document : task.getResult())
                {
                    // Map<String ,Object> map = document.getData();
                    Song song = document.toObject(Song.class);
                    songs.addSongs(song);
                }
                // writeAllData(list);
            }
            else
            {
                {
            }
        }
    });
}

public static void readAllData()
{
    ArrayList<Map> list = new ArrayList<>();
    songs = new Songs();
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("Master").get().addOnCompleteListener(new
OnCompleteListener<QuerySnapshot>()
    {
        @Override
        public void onComplete(@NonNull Task<QuerySnapshot> task)
        {
            if(task.isSuccessful())
            {
                for(QueryDocumentSnapshot document : task.getResult())
                {
                    Map<String ,Object> map = document.getData();
                    list.add(map);
                    //songs.addSongs(new Song(map));
                }
            }
        }
    });
}

```

```

        writeAllData(list);
    }
    else
    {
        }
    }
    });
}

public static void writeAllData(ArrayList<Map> list)
{
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    uid = mAuth.getCurrentUser().getUid();
    for (Map map : list)
    {
        db.collection("Users").document(uid).collection("songs")
            .document().set(map).addOnCompleteListener(new
OnCompleteListener()
        {
            @Override
            public void onComplete(@NonNull Task task)
            {
                }
            }
        });
    }
}

//add all data to master
public static void writeAllData(Songs songs)
{
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    for(int i = 0; i < songs.getAll().size() ;i++)
    {
        Song song = (Song) songs.getAll().get(i);
        Map<String ,Object> map = song.getMap();
        db.collection("Master")
            .document().set(map).addOnCompleteListener(new
OnCompleteListener()
        {
            @Override
            public void onComplete(@NonNull Task task)
            {
                }
            }
        });
    }
}

```

מסד נתונים

🏠 > Users > 3jjxwcbwKhfP1... ✎ More in Google Cloud		
(default)	Users	3jjxwcbwKhfP1cmL4Xv7p4Fvgr32
+ Start collection	+ Add document	+ Start collection
Master	3jjxwcbwKhfP1cmL4Xv7p4Fvgr32	songs
Users	kpK1gXk7HhU8Kgjiu4Ss4IMdBB03	+ Add field
	rWzIbs65k3fy71tq3tVCbaAdL0u1	

🏠 > Users > 3jjxwcbwKhfP1... > songs More in Google Cloud		
Users	3jjxwcbwKhfP1cmL4Xv7p4Fvgr32	songs
+ Add document	+ Start collection	+ Add document
3jjxwcbwKhfP1cmL4Xv7p4Fvgr32	songs	02dziu8gWZ6J6bw20Me5
kpK1gXk7HhU8Kgjiu4Ss4IMdBB03	+ Add field	05zo3NqMC6ajtp0mghXl
rWzIbs65k3fy71tq3tVCbaAdL0u1		18tvCA4KR6zHw0qBRUuj
		2gMb42Zf5eaaSHFZGvty
		2kGqXcbvL1P21Ap0kLxr
		2zB4zFwckvhNUiLVLaDY
		3Jn5L0Gw1uGhF65grrb3
		57RYcAGfQmd3hGfG0LcY

Guess The Song

ניר שדה

🏠 > Users > 3jjxwcbwKhfP1cmL4Xv7p4Fvgr32 > songs > 18tvCA4KRGzH... ✎ More in Google Cloud ▾		
📁 3jjxwcbwKhfP1cmL4Xv7p4Fvgr32	📁 songs	📁 18tvCA4KRGzHwOqBRUuj
+ Start collection	+ Add document	+ Start collection
songs >	02dziu8gMZ6J6bw20Me5	+ Add field
	05zo3NqMC6ajtp0mghX1	artist: "Coldplay"
	18tvCA4KRGzHwOqBRUuj >	fileAudioName: "adventure_of_a_lifetime"
+ Add field	2gMb42Zf5eaaSHFZGvty	genre: "POP ROCK"
	2kGqXcbvL1P21Ap0kLxr	isGuessed: false
	2zB4zFwckvhNUiLVLaDY	lyrics: "Turn your magic on, to me she'd say Everything you want's a dream away"
	3Jn5L0Gw1u6hF65grrb3	name: "Adventure of a Lifetime"
	57RYcAGfQmd3h6fG0LcY	year: 2015
	5KSU711VGEttamNnFGvK	

מדריך למשתמש

הפרויקט פותח על גרסת PIXEL API 36 בתוכנה Android studio. בפעם הראשונה שמשתמש נכנס לאפליקציה עליו ליצור חשבון משתמש משלו על ידי הקלדת שם משתמש וסיסמה, במידה וחלק מהפרטים יהיו שגויים האפליקציה לא תיתן למשתמש להירשם ולשחק ותוצג הודעה בהתאם ובפעם הבאה שהמשתמש יכנס לאפליקציה הוא לא צריך ליצור משתמש חדש משום שהחשבון שהוא יצר נשמר בבסיס הנתונים של האפליקציה אלא רק להקליד את שם המשתמש והסיסמה שהוא כבר יצר.

לאחר מכן, על המשתמש לבחור את הקטגוריה שלפיה הוא רוצה לנחש שירים והאם הוא רוצה לנחש על פי מילים או אודיו. אחר כך הוא יועבר למסך שבו עליו לנחש את השירים כאשר מוצגת הודעה במידה והוא ינחש את שם השיר כראוי ובין אם הוא יטעה או ששם השיר יוצג בהודעה אם יחליט לוותר על לנחש את השיר הנוכחי, על המשתמש להמשיך לנחש שירים עד שהוא מסיים לנחש את שמות השירים מאותה קטגוריה שהוא בחר בה ותוצג הודעה בהתאם או כאשר ימאס ויחליט לחזור למסך האפשרויות.

בנוסף, למשתמש עומדת האפשרות להיכנס למסך ההגדרות ולהתחיל את המשחק מחדש על ידי לחיצה על הכפתור המיועד לכך ולהגדיר האם הוא מעוניין שהמכשיר ירטוט כאשר הוא ינחש נכון שם של שיר, כמו כן, המשתמש גם יכול להחליט האם הוא רוצה שישמע סאונד לאחר הכניסה לאפליקציה.

רפלקציה

במהלך העבודה על הפרויקט עברתי תהליך חשוב שקידם את היכולות והידע שלי במדעי המחשב, ובנוסף, פיתחתי את היכולת שלי למצוא פתרונות באופן עצמאי לבעיות שעלו במהלך העבודה.

בחרתי ליצור משחק שעוסק במוזיקה משום שאני מאוד אוהב לשמוע ז'אנרים מגוונים של מוזיקה ורציתי ליצור משחק שיאתגר את הידע שלי בנושא שבאמצעותו אוכל להנות.

האתגרים בהם נתקלתי כללו בין היתר את העלאת המידע והוצאת המידע ממסד הנתונים ב-Firebase, על מנת שהמשחק יתפקד כראוי משום שזאת הייתה הפעם הראשונה שבה אני עובד עם הטכנולוגיה הזאת, לשמחתי, נעזרתי בעמיתים לכיתה שגם השתמשו בטכנולוגיה הזאת ושאלתי מהם רעיונות כיצד אני צריך ליצור את מסד הנתונים. בנוסף, נתקלתי בקשיים גם כשהעלתי את קבצי האודיו למשחק משום שבעקבות כך שנתתי לקבצי האודיו שמות לא תקינים, המשחק לא השמיע את אותם קבצים, לשמחתי, הצלחתי לבצע התאמה בין שמות הקבצים לשמות ששמורים במסד הנתונים. בנוסף, רציתי שלמשחק תהיה אפשרות לנחש לפי עשורים, אך הדבר לא יצא לפועל משום שכאשר ניסית ליצור פעולה שתשלף ממסד נתונים שירים בטווח שנים מסוים, הפעולה לא הצליחה לקחת אף שאר מטווח השנים שרציתי ובמקום זאת החלטתי שהמשחק יוכל לבחור מאיזו שנה הוא ירצה לנחש שירים ויצרתי רשימה של כל השנים החל מ-1970 ועד 2019 והפעולה אכן הצליחה להוציא ממסד הנתונים שירים משנה ספציפית.

כמו כן, היה עלי להוריד את כל השירים במסך ולערוך אותם כך שקטעי האודיו ששומעים במשחק תואמים במלל למילות השירים ששמורות במסד הנתונים.

יתר על כן, במהלך יצירת המסך שבו על השחקן לבחור את הערכים שלפיהם הוא רוצה לנחש שירים, נעזרתי בסרטונים ביוטיוב שמסבירים איך ליצור רשימה של אופציות שמהן ניתן לבחור אופציה מסוימת, כמו כן, כאשר נתקלתי בבעיות באמולטור בתוכנת אנדרואיד סטודיו פניתי ל-Chat GPT לקבלת עזרה או שלחילופין נעזרתי בסרטונים ביוטיוב.

להמשך, אקח איתי בעיקר את הכלים והאמצעים איתם נעזרתי כדי להתמודד עם אתגרים ובעיות כדוגמת חיפוש אחר תקלות בתוכנה בסרטוני יוטיוב וחיפוש אחר רעיונות שעשויים לשפר את הפרוייקטים שלי.

בראייה לאחור, הייתי רוצה לאפשר לבחור עוד אמנים או ז'אנרים שלפיהם ניתן לנחש שירים או לחילופין, למצוא דרך קלה ומהירה יותר שבאמצעותה ניתן יהיה להעלות קבצי אודיו למשחק בהתאם לשירים שקיימים במסד הנתונים, כמו כן, הייתי מעוניין שתהיה לשחקן אפשרות לראות את הסטטיסטיקה על הביצועים שלו לדוגמה: כמות השירים מכל ערך שהוא כבר ניחש. בנוסף, הייתי מעוניין להתמצא יותר בתוכנת אנדרואיד סטודיו בעיקר בחלק של הרצת הפרויקט על האמולטור משום שבמהלך העבודה נתקלתי בלא מעט בעיות כאשר הרצתי את הפרויקט על האמולטור.

ביבליוגרפיה

- <https://developer.android.com/studio?hl=he> - אנדרואיד סטודיו
- [/https://chatgpt.com](https://chatgpt.com) - צ'אט ג'י פי טי
- סרטון הסבר על MediaPlayer
https://youtu.be/MG21u819x6U?si=hBW06WMUjm1t7_bS
- סרטון הסבר על DropDown Menu
<https://www.youtube.com/watch?v=jXSNObmB7u4>
- סרטון הסבר על Text Spinner
<https://www.youtube.com/watch?v=PjW-XiQ6usI>
- סרטון הסבר על יצירת לוגו לאפליקציה
<https://youtu.be/m6qBOTjZ4Lw?si=AgMYDwy50fNt7wix>

נספחים

```
package com.nirs.guessthesong;

import android.content.Intent;
import android.os.Bundle;
import android.os.CountDownTimer;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;

public class MainActivity extends AppCompatActivity {
    private static FirebaseAuth mAuth;
    private ImageView img;
    private EditText eUser;
    private EditText ePass;
    private Songs songs = new Songs();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button btn = findViewById(R.id.btnLgn);
        SoundService.setValue(this);

        btn.setOnLongClickListener(new View.OnLongClickListener() {
            @Override
            public boolean onLongClick(View view) {
                String mail = "b@gmail.com";
                String password = "123123";
                login(mail, password);
                return true;
            }
        });
    }
}
```

```
    }  
    });  
  
    eUser = findViewById(R.id.eUserName);  
    ePass = findViewById(R.id.eUserPassword);  
    mAuth = FirebaseAuth.getInstance();  
  
    img = findViewById(R.id.imgLogo);  
    img.setY(1000f);  
    timer.start();  
}  
  
float alpha = 0f;  
CountDownTimer timer = new CountDownTimer(4000, 100) {  
    @Override  
    public void onTick(long l) {  
        alpha += +10f;  
        img.setY(alpha);  
    }  
  
    @Override  
    public void onFinish() {  
        img.setScaleX(1);  
        img.setScaleY(1);  
    }  
};  
  
private void login(String mail, String password) {  
  
    String n = eUser.getText().toString();  
    String p = ePass.getText().toString();  
  
    mAuth.signInWithEmailAndPassword(mail,  
password).addOnCompleteListener(new OnCompleteListener<AuthResult>() {  
        @Override  
        public void onComplete(@NonNull Task<AuthResult> task) {  
            if (task.isSuccessful()) {  
                FirebaseLogic.init(MainActivity.this);  
                FirebaseLogic.readUserData();  
                Intent intent = new Intent(MainActivity.this, OptionsActivity.class);  
                SoundService.playSoundOk();  
            }  
        }  
    });  
}
```

```
        startActivity(intent);
    } else {
        SoundService.playSoundWrong();
        Toast.makeText(MainActivity.this, "Fail: " + task.getException(),
Toast.LENGTH_SHORT).show();
    }
}
});
}

public void login(View v) {

    String n = eUser.getText().toString().trim();
    String p = ePass.getText().toString().trim();

    if (n.isEmpty() || p.isEmpty())
    {
        SoundService.playSoundWrong();
        Toast.makeText(MainActivity.this, "Please enter both email and password",
Toast.LENGTH_SHORT).show();
    }
    else
    {
        login(n, p);
    }
}

public void createNewAccount(View v)
{
    String mail = eUser.getText().toString().trim();
    String password = ePass.getText().toString().trim();
    if (mail.isEmpty() || password.isEmpty()) {
        Toast.makeText(MainActivity.this, "Please enter both email and password",
Toast.LENGTH_SHORT).show();
    }
    else
    {
        songs.create();
    }
}
```

```
mAuth.createUserWithEmailAndPassword(mail,
password).addOnCompleteListener(new OnCompleteListener<AuthResult>() {
    @Override
    public void onComplete(@NonNull Task<AuthResult> task)
    {
        if (task.isSuccessful())
        {
            FirebaseLogic.init(MainActivity.this);
            /*
            for creating the database for the first time
            */
            //FirebaseLogic.writeAllData(songs);
            FirebaseLogic.readAllData();

            SoundService.playSoundOk();
            Toast.makeText(MainActivity.this, "Welcome",
Toast.LENGTH_SHORT).show();
            Intent intent = new Intent(MainActivity.this, OptionsActivity.class);
            startActivity(intent);
        }
        else
        {
            SoundService.playSoundWrong();
            Toast.makeText(MainActivity.this, "Fail: " + task.getException(),
Toast.LENGTH_SHORT).show();
        }
    }
});
}
```

```
public class OptionsActivity extends AppCompatActivity
{
    private static FirebaseAuth mAuth;
    private Spinner spinner;
    private String selectedCategory;
    private String selectedValue;
    private String isLyricsOrAudio ;
    private Button playBtn;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        SaveData.init(this);
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_game);
        String[] decadeArray = getResources().getStringArray(R.array.years_list);

        selectedCategory = SaveData.returnCategory();
        selectedValue = SaveData.returnValue();
        isLyricsOrAudio = SaveData.returnAudioOrLyrics();

        TextInputLayout textYear = findViewById(R.id.yearOption);
        TextInputLayout textArtist = findViewById(R.id.artistOption);
        TextInputLayout textGenre = findViewById(R.id.genreOption);
        playBtn = findViewById(R.id.playBtn);
        RadioGroup radioGroup = findViewById(R.id.typeOption);
        spinner = findViewById(R.id.spinner);

        MaterialAutoCompleteTextView autoCompleteTextViewGenre =
        findViewById(R.id.genreTV);
        MaterialAutoCompleteTextView autoCompleteTextViewArtist =
        findViewById(R.id.artistTV);
        MaterialAutoCompleteTextView autoCompleteTextViewYear =
        findViewById(R.id.yearTV);

        ArrayAdapter<CharSequence> adapter =
        ArrayAdapter.createFromResource(this, R.array.spinner_list,
        android.R.layout.simple_spinner_item);
```

```
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item
);
    spinner.setAdapter(adapter);

    if (selectedCategory != null && !selectedCategory.isEmpty())
    {
        int spinnerPosition = adapter.getPosition(selectedCategory);
        spinner.setSelection(spinnerPosition);
    }

    if (isLyricsOrAudio.equals("audio"))
    {
        radioGroup.check(R.id.radioButtonAudio);
    }
    else
    {
        radioGroup.check(R.id.radioButtonLyrics);
    }

    radioGroup.setOnCheckedChangeListener(new
RadioGroup.OnCheckedChangeListener()
    {
        @Override
        public void onCheckedChanged(RadioGroup group, int checkedId)
        {
            if (checkedId == R.id.radioButtonAudio)
            {
                isLyricsOrAudio = "audio";
            }
            else if (checkedId == R.id.radioButtonLyrics)
            {
                isLyricsOrAudio = "lyrics";
            }
            SaveData.setAudioOrLyrics(isLyricsOrAudio); // Save radio group choice
        }
    });
```

```
spinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener()
{
    @Override
    public void onItemSelected(AdapterView<?> parentView, View
selectedItemView, int position, long id)
    {
        selectedCategory = parentView.getItemAtPosition(position).toString();
        SaveData.setCategory(selectedCategory);

        switch (position)
        {
            case 0:
                textArtist.setEnabled(false);
                textYear.setEnabled(false);
                textGenre.setEnabled(true);
                autoCompleteTextViewGenre.setText(SaveData.returnGenreValue(),
false);
                break;
            case 1:
                textArtist.setEnabled(true);
                textYear.setEnabled(false);
                textGenre.setEnabled(false);
                autoCompleteTextViewArtist.setText(SaveData.returnArtistValue(),
false);
                break;
            case 2:
                textArtist.setEnabled(false);
                textYear.setEnabled(true);
                textGenre.setEnabled(false);
                autoCompleteTextViewYear.setText(SaveData.returnYearValue(),
false);
                break;
        }
    }
})

@Override
public void onNothingSelected(AdapterView<?> parentView)
{
    textArtist.setEnabled(false);
    textYear.setEnabled(false);
}
```

```
        textGenre.setEnabled(false);
    }
});
```

```
autoCompleteTextViewGenre.setOnItemClickListener((parent, view, position, id)
->
{
    selectedValue = (String) parent.getItemAtPosition(position);
    SaveData.setGenreValue(selectedValue);
    SaveData.setValue(selectedValue);
});
```

```
autoCompleteTextViewArtist.setOnItemClickListener((parent, view, position, id) ->
{
    selectedValue = (String) parent.getItemAtPosition(position);
    SaveData.setArtistValue(selectedValue);
    SaveData.setValue(selectedValue);
});
```

```
autoCompleteTextViewYear.setOnItemClickListener((parent, view, position, id) ->
{
    selectedValue = (String) parent.getItemAtPosition(position);
    SaveData.setYearValue(selectedValue);
    SaveData.setValue(selectedValue);
});
}
```

```
public void goToSettings(View v)
{
    Intent intent = new Intent(this, SettingsActivity.class);
    startActivity(intent);
}
```



```
public void goToPlay(View v)
{
    selectedValue = SaveData.returnValue();
    Intent intent = new Intent(this, PlayActivity.class);
    intent.putExtra("lyricsOrAudio", isLyricsOrAudio);
    intent.putExtra("selectedValue", selectedValue);
    intent.putExtra("selectedCategory", selectedCategory);
    startActivity(intent);
}

public void goToActivity(View v)
{
    Intent intent = new Intent(this, MainActivity.class);
    startActivity(intent);
}
}
```

```
public class PlayActivity extends AppCompatActivity
{
    private String selectedValue;
    private String selectedCategory;
    private String selectedOption;
    private String songName;
    private Vibrator vibrate;

    private EditText eSong;
    private TextView songLyrics;
    private TextView valueTitle;
    private ImageButton btnPlay, btnStop, btnPause;
    private Button btnGuess, btnGiveUp;
    private MediaPlayer player;
    private boolean showTextBox = false;

    private int resAudioFile;
    private int arrayLength;
    private int currSongPos = 0;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        String[] decadeArray = getResources().getStringArray(R.array.years_list);

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_play_activity);
        eSong = findViewById(R.id.eSongName);
        songLyrics = findViewById(R.id.songLyrics);
        valueTitle = findViewById(R.id.valueTitle);
        btnPlay = findViewById(R.id.play);
        btnPause = findViewById(R.id.pause);
        btnStop = findViewById(R.id.stop);
        btnGuess = findViewById(R.id.guessSong);
        btnGiveUp = findViewById(R.id.giveUp);
        vibrate = (Vibrator) getSystemService(Context.VIBRATOR_SERVICE);
        FirebaseLogic.init(PlayActivity.this);

        registerReceiver(receiver, new IntentFilter(FirebaseLogic.LOAD));

        selectedValue = getIntent().getStringExtra("selectedValue");
```

```
selectedCategory = getIntent().getStringExtra("selectedCategory");
selectedOption = getIntent().getStringExtra("lyricsOrAudio");
valueTitle.setText(selectedValue);

if (selectedCategory.equals("Genre"))
{
    FirebaseLogic.getSongsByGenre(selectedValue);
}

else if (selectedCategory.equals("Artist"))
{
    FirebaseLogic.getSongsByArtist(selectedValue);
}

else if(selectedCategory.equals("Year"))
{
    FirebaseLogic.getSongsByYear(Integer.parseInt(selectedValue));
}
}

private BroadcastReceiver receiver = new BroadcastReceiver()
{
    @Override
    public void onReceive(Context context, Intent intent)
    {
        arrayLength = FirebaseLogic.songs.getSize();
        setSong();
        unregisterReceiver(receiver);
    }
};

public void setSong()
{
    if (arrayLength > 0)
    {
        Song song = FirebaseLogic.songs.getSong(currSongPos);
        songLyrics.setText(song.getLyrics());
        songName = song.getName();
    }
}
```

```
resAudioFile = getResources().getIdentifier(song.getFileAudioName(), "raw",  
getPackageName());
```

```
if (selectedOption.equals("lyrics")) {  
    btnPlay.setVisibility(View.GONE);  
    btnStop.setVisibility(View.GONE);  
    btnPause.setVisibility(View.GONE);  
    songLyrics.setVisibility(View.VISIBLE);
```

```
} else {  
    btnPlay.setVisibility(View.VISIBLE);  
    btnStop.setVisibility(View.VISIBLE);  
    btnPause.setVisibility(View.VISIBLE);  
    songLyrics.setVisibility(View.GONE);
```

```
}
```

```
btnPlay.setOnClickListener(new View.OnClickListener() {
```

```
    public void onClick(View v) {  
        if (player == null) {  
            player = MediaPlayer.create(PlayActivity.this, resAudioFile);  
        }  
        player.start();  
    }  
});
```

```
btnPause.setOnClickListener(new View.OnClickListener() {
```

```
    @Override  
    public void onClick(View view) {  
        if (player != null) {  
            player.pause();  
        }  
    }  
});
```

```
btnStop.setOnClickListener(new View.OnClickListener() {
```

```
    @Override  
    public void onClick(View view) {  
        if (player != null) {
```

```
        player.release();
        player = null;
    }
}
});
}
else
{
    btnGiveUp.setEnabled(false);
    btnGuess.setEnabled(false);
    btnPause.setEnabled(false);
    btnPlay.setEnabled(false);
    btnStop.setEnabled(false);
    Toast.makeText(this, "No songs of that category was found",
Toast.LENGTH_SHORT).show();
}
}
```

```
public void goToGame(View v)
{
    if (player != null)
    {
        player.release();
        player = null;
    }

    Intent intent = new Intent(this, OptionsActivity.class);
    startActivity(intent);
}
```

```
public void guessSong(View v)
{
    if (player != null)
    {
        player.release();
        player = null;
    }
}
```

```
String eSongString = eSong.getText().toString().trim();
if (songName.toLowerCase().equalsIgnoreCase(eSongString) ||
songName.toLowerCase().equalsIgnoreCase(eSongString.toLowerCase()) ||
(songName.replace(" ", "").equalsIgnoreCase(eSongString)))
{
    if (SaveData.isVibrate())
    {
        if (android.os.Build.VERSION.SDK_INT >=
android.os.Build.VERSION_CODES.O)
        {
            vibrate.vibrate(VibrationEffect.createOneShot(1000,
VibrationEffect.DEFAULT_AMPLITUDE));
        }
    }

    Toast.makeText(this, "Well Done", Toast.LENGTH_SHORT).show();
    currSongPos++;

    registerReceiver(receiver_two, new IntentFilter(FirebaseLogic.LOAD));
    FirebaseLogic.markSongAsGuessedForUser(songName);

    if (currSongPos < arrayLength)
    {
        setSong();
    }
    else
    {
        Toast.makeText(this, "You finished to guess all the songs of that category",
Toast.LENGTH_SHORT).show();
        btnGiveUp.setEnabled(false);
        btnGuess.setEnabled(false);
        btnPause.setEnabled(false);
        btnPlay.setEnabled(false);
        btnStop.setEnabled(false);
    }
}
else
{
    Toast.makeText(this, "Try again", Toast.LENGTH_SHORT).show();
}
eSong.setText("");
}
```

```
private BroadcastReceiver receiver_two = new BroadcastReceiver()
{
    @Override
    public void onReceive(Context context, Intent intent)
    {
        unregisterReceiver(receiver_two);
    }
};

public void giveUp (View v)
{
    if (player != null)
    {
        player.release();
        player = null;
    }
    Toast.makeText(this, "The song is: " + songName,
Toast.LENGTH_SHORT).show();
    currSongPos++;
    if(currSongPos < arrayLength)
    {
        setSong();
    }
    else
    {
        Toast.makeText(this, "You finished to guess all the songs of that category",
Toast.LENGTH_SHORT).show();
        btnGiveUp.setEnabled(false);
        btnGuess.setEnabled(false);
        btnPause.setEnabled(false);
        btnPlay.setEnabled(false);
        btnStop.setEnabled(false);
    }
    eSong.setText("");
}

public void goToSettings(View v)
{
    Intent intent = new Intent(this, SettingsActivity.class);
```

Guess The
Song

גיר שדה

```
        startActivity(intent);  
    }  
}
```



```
public class FirebaseLogic
{
    public static final String LOAD = "Load songs";
    private static FirebaseAuth mAuth;
    public static Songs songs;
    private static String uid;
    private static Context context;

    public static void init(Context c)
    {
        mAuth = FirebaseAuth.getInstance();
        context = c;
    }

    public static void readUserData()
    {
        uid = mAuth.getCurrentUser().getUid();
        ArrayList<Map> list = new ArrayList<>();
        songs = new Songs();
        FirebaseFirestore db = FirebaseFirestore.getInstance();
        db.collection("Users").document(uid).collection("songs")
            .get().addOnCompleteListener(new
OnCompleteListener<QuerySnapshot>()
        {
            @Override
            public void onComplete(@NonNull Task<QuerySnapshot> task)
            {
                if(task.isSuccessful())
                {
                    for(QueryDocumentSnapshot document : task.getResult())
                    {
                        // Map<String ,Object> map = document.getData();
                        Song song = document.toObject(Song.class);
                        songs.addSongs(song);
                    }
                }
                /*
                used for creating a new database for a user
                */
                // writeAllData(list);
            }
        }
    }
}
```

```

        else
        {

        }

    }

});

}

```

```

public static void readAllData()
{
    ArrayList<Map> list = new ArrayList<>();
    songs = new Songs();
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    db.collection("Master").get().addOnCompleteListener(new
OnCompleteListener<QuerySnapshot>()
    {
        @Override
        public void onComplete(@NonNull Task<QuerySnapshot> task)
        {
            if(task.isSuccessful())
            {
                for(QueryDocumentSnapshot document : task.getResult())
                {
                    Map<String ,Object> map = document.getData();
                    list.add(map);
                    /*
                    used for creating the ArrayList
                    */
                    //songs.addSongs(new Song(map));
                }
                writeAllData(list);
            }
            else
            {

            }
        }
    });
}

}

```

```

public static void writeAllData(ArrayList<Map> list)

```

```
{
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    uid = mAuth.getCurrentUser().getUid();
    for (Map map : list)
    {
        db.collection("Users").document(uid).collection("songs")
            .document().set(map).addOnCompleteListener(new
OnCompleteListener()
            {
                @Override
                public void onComplete(@NonNull Task task)
                {

                }
            });
    }
}
```

```
//add all data to master
public static void writeAllData(Songs songs)
{
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    for(int i = 0; i < songs.getAll().size() ;i++)
    {
        Song song = (Song) songs.getAll().get(i);
        Map<String ,Object> map = song.getMap();
        db.collection("Master")
            .document().set(map).addOnCompleteListener(new
OnCompleteListener()
            {
                @Override
                public void onComplete(@NonNull Task task)
                {

                }
            });
    }
}
```

```
public static Songs getSongs()
```

```
{  
    return songs;  
}
```

```
public static void getSongsByGenre(String genreType)  
{  
    songs = new Songs();  
    FirebaseFirestore db = FirebaseFirestore.getInstance();  
    uid = mAuth.getCurrentUser().getUid();  
  
    db.collection("Users").document(uid).collection("songs")  
        .whereEqualTo("genre", genreType)  
        .whereEqualTo("isGuessed", false)  
        .get()  
        .addOnCompleteListener(new OnCompleteListener<QuerySnapshot>()  
        {  
            @Override  
            public void onComplete(@NonNull Task<QuerySnapshot> task)  
            {  
                if (task.isSuccessful())  
                {  
                    for (QueryDocumentSnapshot document : task.getResult())  
                    {  
                        Song song = document.toObject(Song.class);  
                        songs.addSongs(song);  
                    }  
                    context.sendBroadcast(new Intent(LOAD));  
                }  
                else  
                {  
                    Log.e("FirebaseLogic", "Error fetching songs", task.getException());  
                }  
            }  
        }  
    });  
}
```

```
public static void getSongsByArtist(String artistName)
{
    songs = new Songs();
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    uid = mAuth.getCurrentUser().getUid();

    db.collection("Users").document(uid).collection("songs")
        .whereEqualTo("artist", artistName)
        .whereEqualTo("isGuessed", false)
        .get()
        .addOnCompleteListener(new OnCompleteListener<QuerySnapshot>()
        {
            @Override
            public void onComplete(@NonNull Task<QuerySnapshot> task)
            {
                if (task.isSuccessful())
                {
                    for (QueryDocumentSnapshot document : task.getResult())
                    {
                        Song song = document.toObject(Song.class);
                        songs.addSongs(song);
                    }
                    context.sendBroadcast(new Intent(LOAD));
                }
                else
                {
                    Log.e("FirebaseLogic", "Error fetching songs", task.getException());
                }
            }
        });
}
```

```
public static void getSongsByYear(int year)
{
```

```
songs = new Songs();
FirebaseFirestore db = FirebaseFirestore.getInstance();
uid = mAuth.getCurrentUser().getUid();

db.collection("Users").document(uid).collection("songs")
    .whereEqualTo("isGuessed", false)
    .whereEqualTo("year", year)
    .get()
    .addOnCompleteListener(new OnCompleteListener<QuerySnapshot>()
    {
        @Override
        public void onComplete(@NonNull Task<QuerySnapshot> task)
        {
            if (task.isSuccessful())
            {
                for (QueryDocumentSnapshot document : task.getResult())
                {
                    Song song = document.toObject(Song.class);
                    songs.addSongs(song);
                }
                context.sendBroadcast(new Intent(LOAD));
            }
            else
            {
                Log.e("FirebaseLogic", "Error fetching songs", task.getException());
            }
        }
    });
}
```

```
public static void markSongAsGuessedForUser(String songName)
{
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    uid = mAuth.getCurrentUser().getUid();

    db.collection("Users").document(uid).collection("songs")
        .whereEqualTo("name", songName)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            for (QueryDocumentSnapshot document : queryDocumentSnapshots)
            {
```

```
        db.collection("Users").document(uid)
            .collection("songs").document(document.getId())
            .update("isGuessed", true);
    }
    context.sendBroadcast(new Intent(LOAD));
})
.addOnFailureListener(e -> Log.e("Firestore", "Error fetching song", e));
}

public static void resetGame()
{
    FirebaseFirestore db = FirebaseFirestore.getInstance();
    uid = mAuth.getCurrentUser().getUid();

    db.collection("Users").document(uid).collection("songs")
        .whereEqualTo("isGuessed", true)
        .get()
        .addOnSuccessListener(queryDocumentSnapshots -> {
            for (QueryDocumentSnapshot document : queryDocumentSnapshots)
            {
                db.collection("Users").document(uid)
                    .collection("songs").document(document.getId())
                    .update("isGuessed", false);
            }
            context.sendBroadcast(new Intent(LOAD));
        })
        .addOnFailureListener(e -> Log.e("Firestore", "Error fetching song", e));
    }
}
```

```
public class SaveData
{
    private static SharedPreferences sp;
    private static SharedPreferences.Editor edit;

    private static final String YEAR_VALUE_KEY = "year";
    private static final String GENRE_VALUE_KEY = "genre";
    private static final String ARTIST_VALUE_KEY = "artist";
    private static final String VALUE_KEY = "value";
    private static final String VIBRATE_KEY = "vibrate";
    private static final String CATEGORY_KEY = "category";
    private static final String AUDIO_OR_LYRICS_KEY = "audioOrLyrics";

    private static String category = " ";
    private static String year_value = " ";
    private static String genre_value = " ";
    private static String artist_value = " ";
    private static String value = " ";
    private static String audioOrLyrics = "audio";
    private static boolean vibrate;

    public static void init(Context context)
    {
        sp = context.getSharedPreferences("data",Context.MODE_PRIVATE);
        edit = sp.edit();
        category = sp.getString(CATEGORY_KEY, "");
        artist_value = sp.getString(ARTIST_VALUE_KEY, "");
        year_value = sp.getString(YEAR_VALUE_KEY, "");
        genre_value = sp.getString(GENRE_VALUE_KEY, "");

        audioOrLyrics = sp.getString(AUDIO_OR_LYRICS_KEY, "audio");
        vibrate = sp.getBoolean(VIBRATE_KEY,true);
    }

    public static String returnYearValue() {
        return year_value;
    }

    public static void setYearValue(String value)
```



```
{
    SaveData.year_value = value;
    edit.putString(YEAR_VALUE_KEY,value);
    edit.apply();
}

public static String returnGenreValue() {
    return genre_value;
}

public static void setGenreValue(String value)
{
    SaveData.genre_value = value;
    edit.putString(GENRE_VALUE_KEY,value);
    edit.apply();
}

public static String returnArtistValue() {
    return artist_value;
}

public static void setArtistValue(String value)
{
    SaveData.artist_value = value;
    edit.putString(ARTIST_VALUE_KEY,value);
    edit.apply();
}

public static String returnValue()
{
    if(category.equals("Genre"))
    {
        value = returnGenreValue();
    }

    if(category.equals("Year"))
    {
        value = returnYearValue();
    }
    if(category.equals("Artist"))
```

```
{
    value = returnArtistValue();
}

return value;
}

public static void setValue(String value)
{
    SaveData.value = value;
    edit.putString(VALUE_KEY,value);
    edit.apply();
}

public static String returnCategory() {
    return category;
}

public static void setCategory(String category)
{
    SaveData.category = category;
    edit.putString(CATEGORY_KEY, category);
    edit.apply();
}

public static String returnAudioOrLyrics()
{
    return audioOrLyrics;
}

public static void setAudioOrLyrics(String choice)
{
    SaveData.audioOrLyrics = choice;
    edit.putString(AUDIO_OR_LYRICS_KEY, choice);
    edit.apply();
}

public static boolean isVibrate() {
    return vibrate;
}
```

```
}  
  
public static void setVibrate(boolean vibrate) {  
    SaveData.vibrate = vibrate;  
    edit.putBoolean(VIBRATE_KEY,vibrate);  
    edit.commit();  
}  
}
```

```
public class SettingsActivity extends AppCompatActivity
{

    private Switch sVibrate;

    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_settings);
        SaveData.init(this);
        sVibrate = findViewById(R.id.sVibrate);
        sVibrate.setChecked(SaveData.isVibrate());
        FirebaseLogic.init(SettingsActivity.this);
    }

    public void changeVibrate(View v)
    {
        if(sVibrate.isChecked())
        {
            Toast.makeText(this, "Vibrate is ON", Toast.LENGTH_SHORT).show();
            SaveData.setVibrate(sVibrate.isChecked());
        }
        else
        {
            Toast.makeText(this, "Vibrate is OFF", Toast.LENGTH_SHORT).show();
            SaveData.setVibrate(sVibrate.isChecked());
        }
    }

    public void resetBtn(View v)
    {
        registerReceiver(receiver, new IntentFilter(FirebaseLogic.LOAD));
        FirebaseLogic.resetGame();
        Toast.makeText(this, "Game reseted", Toast.LENGTH_SHORT).show();
    }
}
```

```
private BroadcastReceiver receiver = new BroadcastReceiver()
{
    @Override
    public void onReceive(Context context, Intent intent)
    {
        unregisterReceiver(receiver);
    }
};

public void goToGame(View v)
{
    Intent intent = new Intent(this, OptionsActivity.class);
    startActivity(intent);
}
};
```

Guess The Song

גיר שדה

```
public class Song
{
    private String name;
    private int year;
    private String genre;
    private String fileAudioName;
    private String lyrics;
    private String artist;
    private boolean isGuessed;

    public Song(String name, int year, String genre, String fileAudioName, String lyrics,
String artist)
    {
        this.genre = genre;
        this.year = year;
        this.name = name;
        this.fileAudioName = fileAudioName;
        this.lyrics = lyrics;
        this.artist = artist;
        this.isGuessed = false;
    }

    public Song(String name, int year, String genre, String fileAudioName, String lyrics,
String artist, boolean isGuessed)
    {
        this.genre = genre;
        this.year = year;
        this.name = name;
        this.fileAudioName = fileAudioName;
        this.lyrics = lyrics;
        this.artist = artist;
        this.isGuessed = isGuessed;
    }

    public Song()
    {

    }

    public Song(Map<String, Object> map)
    {
        this.name = (String) map.get("name");
```

```
    this.year = (int) map.get("year");
    this.genre = (String) map.get("genre");
    this.fileAudioName = (String) map.get("fileAudioName");
    this.lyrics = (String) map.get("lyrics");
    this.artist = (String) map.get("artist");
    this.isGuessed = (boolean) map.get("isGuessed");
}
```

```
public Map<String, Object> getMap()
{
    Map<String, Object> map = new HashMap<>();
    map.put("name" , name);
    map.put("year" , year);
    map.put("genre" , genre);
    map.put("fileAudioName" , (fileAudioName.toLowerCase()).replace(" ", "_"));
    map.put("lyrics" , lyrics);
    map.put("artist" , artist);
    map.put("isGuessed" , isGuessed);
    return map;
}
```

```
public void setFileAudioName(String fileAudioName) {this.fileAudioName =
fileAudioName;}
```

```
public void setName(String name) {this.name = name;}
```

```
public void setYear(int year) {this.year = year;}
```

```
public void setGenre(String genre)
{
    this.genre = genre;
}
```

```
public void setArtist(String artist)
{
    this.artist = artist;
}
```

```
public void setIsGuessed(boolean isGuessed){this.isGuessed = isGuessed;}
```

```
public String getFileAudioName()
{
```

```
        return this.fileName;
    }

    public String getName()
    {
        return this.name;
    }

    public int getYear()
    {
        return this.year;
    }

    public String getGenre()
    {
        return this.genre;
    }

    public String getArtist()
    {
        return this.artist;
    }

    public boolean getIsGuessed() {return this.isGuessed;}

    public String getLyrics() {
        return lyrics;
    }

    public void setLyrics(String lyrics) {
        this.lyrics = lyrics;
    }

    public boolean isGuessed() {
        return isGuessed;
    }

    public void setGuessed(boolean guessed) {
        isGuessed = guessed;
    }
}
```



```
public class Songs {
    private ArrayList<Song> songs;

    public Songs()
    {
        this.songs = new ArrayList<>();
    }

    public void addSongs(Song s) {
        songs.add(s);
    }

    public void removeSong(int number) {songs.remove(number);}

    public ArrayList<Song> getAll() {
        return songs;
    }

    public Song getSong(int number) {
        return songs.get(number);
    }

    public int getSize()
    {
        return songs.size();
    }

    public void create() {
        songs.add(new Song("Space Oddity", 1969, "ROCK", "Space Oddity", "Ground
control to Major Tom\nTake your protein pills and put your helmet on", "David Bowie"));
        songs.add(new Song("Starman", 1972, "ROCK", "Starman", "Didn't know what
time it was, the lights were low\nI leaned back on my radio\nSome cat was layin' down
some rock 'n' roll\nLotta soul, he said", "David Bowie"));
        songs.add(new Song("Let's Dance", 1983, "ROCK", "Lets Dance", "Let's
dance\nPut on your red shoes and dance the blues", "David Bowie"));

        songs.add(new Song("Wonderwall", 1995, "BRITPOP", "Wonderwall", "Today is
gonna be the day that they're gonna throw it back to you\nBy now, you should've
somehow realised what you gotta do\nI don't believe that anybody feels the way I do
about you now", "Oasis"));
    }
```

```
songs.add(new Song("Supersonic", 1994, "BRITPOP", "Supersonic", "I need to  
be myself\nI can't be no one else", "Oasis"));
```

```
songs.add(new Song("Don't Look Back in Anger", 1995, "BRITPOP", "Dont Look  
Back in Anger", "Slip inside the eye of your mind\nDon't you know you might\nfind A  
better place to play", "Oasis"));
```

```
songs.add(new Song("Live Forever", 1994, "BRITPOP", "Live Forever", "Maybe I  
don't really want to know\nHow your garden grows, 'cause I just want to fly", "Oasis"));
```

```
songs.add(new Song("Lithium", 1991, "GRUNGE", "Lithium", "I'm so happy  
'cause today I've found my friends\nThey're in my head", "Nirvana"));
```

```
songs.add(new Song("Come as You Are", 1991, "GRUNGE", "Come as You Are",  
"Come as you are, as you were\nAs I want you to be", "Nirvana"));
```

```
songs.add(new Song("Smells Like Teen Spirit", 1991, "GRUNGE", "Smells Like  
Teen Spirit", "Load up on guns, bring your friends\nIt's fun to lose and to pretend",  
"Nirvana"));
```

```
songs.add(new Song("About a Girl", 1989, "GRUNGE", "About a Girl", "I need an  
easy friend\nI do, with an ear to lend", "Nirvana"));
```

```
songs.add(new Song("Bohemian Rhapsody", 1975, "ROCK", "Bohemian  
Rhapsody", "Is this the real life?\nIs this just fantasy?\nCaught in a landslide, no  
escape from reality", "Queen"));
```

```
songs.add(new Song("We Will Rock You", 1977, "ROCK", "We Will Rock You",  
"Buddy, you're a boy, make a big noise\nPlayin' in the street, gonna be a big man  
someday", "Queen"));
```

```
songs.add(new Song("The Show Must Go On", 1991, "ROCK", "The Show Must  
Go On", "Empty spaces, what are we living for\nAbandoned places, I guess we know  
the score", "Queen"));
```

```
songs.add(new Song("Don't Stop Me Now", 1978, "ROCK", "Dont Stop Me Now",  
"Tonight I'm gonna have myself a real good time\nI feel alive", "Queen"));
```

```
songs.add(new Song("I Want to Break Free", 1984, "ROCK", "I Want to Break  
Free", "I want to break free\nI want to break free", "Queen"));
```

```
songs.add(new Song("Hammer to Fall", 1984, "ROCK", "Hammer to Fall", "Here  
we stand or here we fall\nHistory won't care at all", "Queen"));
```

```
songs.add(new Song("We Are the Champions", 1977, "ROCK", "We Are the  
Champions", "I've paid my dues\nTime after time", "Queen"));
```

```
songs.add(new Song("Wish You Were Here", 1975, "PROGRESSIVE ROCK",  
"Wish You Were Here", "So, so you think you can tell\nHeaven from Hell?", "Pink  
Floyd"));
```

```
songs.add(new Song("Comfortably Numb", 1979, "PROGRESSIVE ROCK",  
"Comfortably Numb", "Hello?\nIs there anybody in there?\nJust nod if you can hear  
me", "Pink Floyd"));
```

```
songs.add(new Song("Time", 1973, "PROGRESSIVE ROCK", "Time", "Ticking  
away the moments that make up a dull day\nYou fritter and waste the hours in an  
offhand way", "Pink Floyd"));  
songs.add(new Song("Another Brick in the Wall", 1979, "PROGRESSIVE  
ROCK", "Another Brick in the Wall", "We don't need no education\nWe don't need no  
thought control", "Pink Floyd"));  
songs.add(new Song("Shine On You Crazy Diamond", 1975, "PROGRESSIVE  
ROCK", "Shine On You Crazy Diamond", "Remember when you were young\nYou  
shone like the sun", "Pink Floyd"));  
songs.add(new Song("Money", 1973, "PROGRESSIVE ROCK", "Money",  
"Money\nGet away\nYou get a good job with more pay and you're okay", "Pink  
Floyd"));  
songs.add(new Song("Hey You", 1979, "PROGRESSIVE ROCK", "Hey You",  
"Hey you, out there in the cold\nGetting lonely, getting old\nCan you feel me?", "Pink  
Floyd"));  
songs.add(new Song("Us and Them", 1973, "PROGRESSIVE ROCK", "Us and  
Them", "Us, and them\nAnd after all, we're only ordinary men", "Pink Floyd"));  
songs.add(new Song("High Hopes", 1994, "PROGRESSIVE ROCK", "High  
Hopes", "Beyond the horizon of the place we lived when we were young\nIn a world of  
magnets and miracles", "Pink Floyd"));  
songs.add(new Song("Breathe", 1973, "PROGRESSIVE ROCK", "Breathe",  
"Breathe, breathe in the air\nDon't be afraid to care", "Pink Floyd"));  
  
songs.add(new Song("Back in Black", 1980, "ROCK", "Back in Black", "Back in  
black\nI hit the sack\nI've been too long, I'm glad to be back", "AC/DC"));  
songs.add(new Song("Highway to Hell", 1979, "ROCK", "Highway to Hell", "Living  
easy, living free\nSeason ticket on a one-way ride", "AC/DC"));  
songs.add(new Song("Thunderstruck", 1990, "ROCK", "Thunderstruck", "I was  
caught\nIn the middle of a railroad track", "AC/DC"));  
  
songs.add(new Song("Sunday Bloody Sunday", 1983, "ROCK", "Sunday Bloody  
Sunday", "I can't believe the news today\nOh, I can't close my eyes and make it go  
away", "U2"));  
songs.add(new Song("One", 1991, "ROCK", "One", "Is it getting better\nOr do  
you feel the same?\nWill it make it easier on you\nNow you got someone to blame?",  
"U2"));  
songs.add(new Song("Pride", 1984, "ROCK", "Pride", "One man come in the  
name of love\nOne man come and go", "U2"));  
songs.add(new Song("Mysterious Ways", 1991, "ROCK", "Mysterious Ways",  
"Johnny take a walk with your sister the moon\nLet her pale light in to fill up your  
room", "U2"));
```

```
songs.add(new Song("Beautiful Day", 2000, "ROCK", "Beautiful Day", "The heart  
is a bloom\nShoots up through the stony ground", "U2"));
```

```
songs.add(new Song("With or Without You", 1987, "ROCK", "With or Without  
You", "See the stone set in your eyes\nSee the thorn twist in your side", "U2"));
```

```
songs.add(new Song("Californication", 1999, "ALTERNATIVE ROCK",  
"Californication", "Psychic spies from China\nTry to steal your mind's elation", "Red  
Hot Chili Peppers"));
```

```
songs.add(new Song("Under the Bridge", 1992, "ALTERNATIVE ROCK", "Under  
the Bridge", "Sometimes I feel like I don't have a partner\nSometimes I feel like my  
only friend", "Red Hot Chili Peppers"));
```

```
songs.add(new Song("Scar Tissue", 1999, "ALTERNATIVE ROCK", "Scar  
Tissue", "Scar tissue that I wish you saw\nSarcastic mister know-it-all", "Red Hot Chili  
Peppers"));
```

```
songs.add(new Song("Otherside", 1999, "ALTERNATIVE ROCK", "Otherside",  
"How long, how long will I slide?\nSeparate my side, I don't", "Red Hot Chili  
Peppers"));
```

```
songs.add(new Song("By the Way", 2002, "ALTERNATIVE ROCK", "By the Way",  
"Standing in line\nTo see the show tonight", "Red Hot Chili Peppers"));
```

```
songs.add(new Song("Dani California", 2006, "ALTERNATIVE ROCK", "Dani  
California", "Gettin' born in the state of Mississippi\nPapa was a copper and mama  
was a hippie", "Red Hot Chili Peppers"));
```

```
songs.add(new Song("Can't Stop", 2002, "ALTERNATIVE ROCK", "Cant Stop",  
"Can't stop, addicted to the shindig\nChop top, he says I'm gonna win big", "Red Hot  
Chili Peppers"));
```

```
songs.add(new Song("Snow", 2006, "ALTERNATIVE ROCK", "Snow", "Come to  
decide that the things that I tried\nWere in my life just to get high on", "Red Hot Chili  
Peppers"));
```

```
songs.add(new Song("Give It Away", 1991, "FUNK ROCK", "Give It Away",  
"What I got, you gotta give it to your mama\nWhat I got, you gotta give it to your papa",  
"Red Hot Chili Peppers"));
```

```
songs.add(new Song("Sweet Child o' Mine", 1987, "ROCK", "Sweet Child o  
Mine", "She's got a smile that it seems to me\nReminds me of childhood memories",  
"Guns N' Roses"));
```

```
songs.add(new Song("Welcome to the Jungle", 1987, "HARD ROCK", "Welcome  
to the Jungle", "Welcome to the jungle, we got fun and games", "Guns N' Roses"));
```

```
songs.add(new Song("Paradise City", 1987, "ROCK", "Paradise City", "Take me  
down to the paradise city,\nWhere the grass is green and the girls are pretty", "Guns  
N' Roses"));
```

```
songs.add(new Song("Don't Cry", 1991, "ROCK", "Dont Cry", "Talk to me softly,  
there's something in your eyes\nDon't hang your head in sorrow and please don't cry",  
"Guns N' Roses"));
```

```
songs.add(new Song("November Rain", 1991, "ROCK", "November Rain", "When  
I look into your eyes\nI can see a love restrained", "Guns N' Roses"));

songs.add(new Song("Yellow", 2000, "ALTERNATIVE ROCK", "Yellow", "Look at  
the stars, look how they shine for you", "Coldplay"));
songs.add(new Song("Fix You", 2005, "ALTERNATIVE ROCK", "Fix You", "When  
you try your best, but you don't succeed", "Coldplay"));
songs.add(new Song("Viva La Vida", 2008, "ALTERNATIVE ROCK", "Viva La  
Vida", "I used to rule the world\nSeas would rise when I gave the word", "Coldplay"));
songs.add(new Song("Clocks", 2002, "ALTERNATIVE ROCK", "Clocks", "The  
lights go out and I can't be saved", "Coldplay"));
songs.add(new Song("The Scientist", 2002, "ALTERNATIVE ROCK", "The  
Scientist", "Come up to meet you, tell you I'm sorry", "Coldplay"));
songs.add(new Song("Paradise", 2011, "ALTERNATIVE ROCK", "Paradise",  
"When she was just a girl\nShe expected the world", "Coldplay"));
songs.add(new Song("Adventure of a Lifetime", 2015, "POP ROCK", "Adventure  
of a Lifetime", "Turn your magic on, to me she'd say\nEverything you want's a dream  
away", "Coldplay"));
songs.add(new Song("A Sky Full of Stars", 2014, "POP ROCK", "A Sky Full of  
Stars", "'Cause you're a sky, 'cause you're a sky full of stars", "Coldplay"));
songs.add(new Song("Magic", 2014, "POP ROCK", "Magic", "Call it magic, call it  
true\nI call it magic when I'm with you", "Coldplay"));
}
```

```
package com.nirs.guessthesong;

import android.content.Context;
import android.media.AudioManager;
import android.media.SoundPool;
import android.os.Build;

public class SoundService
{
    private static SoundPool soundpool;
    private static float volume;
    public static int playOk;
    public static int playWrong;
    public static int playWin;

    public static void setValue(Context context)
    {
        if(Build.VERSION.SDK_INT >= Build.VERSION_CODES.LOLLIPOP)
        {
            soundpool = new SoundPool.Builder().setMaxStreams(20).build();
        }
        else
        {
            soundpool = new SoundPool(20, AudioManager.STREAM_MUSIC, 1);
        }
        volume = 1;
        playOk = soundpool.load(context,R.raw.entry,1);
        playWrong = soundpool.load(context,R.raw.fail, 1);
    }

    public static void playSoundWrong()
    {
        if(SaveData.isSound())
        {
            soundpool.play(playWrong, volume, volume, 1, 0, 1);
        }
    }

    public static void playSoundOk()
    {
        if(SaveData.isSound())
        {
            soundpool.play(playOk,volume,volume,1,0,1);
        }
    }
}
```

Guess The
Song

גיר שדה

}

}

}


```
<?xml version="1.0" encoding="utf-8"?>
```

```
<resources>
```

```
  <string-array name="spinner_list">
```

```
    <item>Genre</item>
```

```
    <item>Artist</item>
```

```
    <item>Year</item>
```

```
  </string-array>
```

```
  <string-array name="years_list">
```

```
    <item>1970</item>
```

```
    <item>1971</item>
```

```
    <item>1972</item>
```

```
    <item>1973</item>
```

```
    <item>1974</item>
```

```
    <item>1975</item>
```

```
    <item>1976</item>
```

```
    <item>1977</item>
```

```
    <item>1978</item>
```

```
    <item>1979</item>
```

```
    <item>1980</item>
```

```
    <item>1981</item>
```

```
    <item>1982</item>
```

```
    <item>1983</item>
```

```
    <item>1984</item>
```

```
    <item>1985</item>
```

```
    <item>1986</item>
```

```
    <item>1987</item>
```

```
    <item>1988</item>
```

```
    <item>1989</item>
```

```
    <item>1990</item>
```

```
    <item>1991</item>
```

```
    <item>1992</item>
```

```
    <item>1993</item>
```

```
    <item>1994</item>
```

```
    <item>1995</item>
```

```
    <item>1996</item>
```

```
    <item>1997</item>
```

```
    <item>1998</item>
```

```
    <item>1999</item>
```

```
    <item>2000</item>
```

```
<item>2001</item>
<item>2002</item>
<item>2003</item>
<item>2004</item>
<item>2005</item>
<item>2006</item>
<item>2007</item>
<item>2008</item>
<item>2009</item>
<item>2010</item>
<item>2011</item>
<item>2012</item>
<item>2013</item>
<item>2014</item>
<item>2015</item>
<item>2016</item>
<item>2017</item>
<item>2018</item>
<item>2019</item>
```

```
</string-array>
```

```
<string-array name="artists_list">
  <item>Queen</item>
  <item>Oasis</item>
  <item>AC/DC</item>
  <item>Nirvana</item>
  <item>Guns N' Roses</item>
  <item>Pink Floyd</item>
  <item>U2</item>
  <item>David Bowie</item>
  <item>Red Hot Chili Peppers</item>
  <item>Coldplay</item>
</string-array>
```

```
<string-array name="genre_list">
  <item>ROCK</item>
  <item>POP ROCK</item>
  <item>ALTERNATIVE ROCK </item>
  <item>BRITPOP</item>
```

Guess The
Song

גיר שדה

```
<item>GRUNGE</item>  
</string-array>
```

```
</resources>
```